UNIVERSITY PARTNER

UNIVERSITY OF WOLVERHAMPTON

HERALD COLLEGE KATHMANDU

Flower Image Classification using Convolutional Neural Networks and Transfer Learning

Github Link- https://github.com/Rashidsid/aiml-workshop/blob/main/Final_Assessment/Flower_Classification_Assessment.ipynb

Student Id :2413748

Student Name :Mohammad Rashid Siddiqui

Group :L6CG10

Module Leader :Mr. Siman Giri

Tutor :Mr. Anish Khatiwoda

Submitted on :May 08, 2026

Abstract

This report presents the implementation and evaluation of the Part II Computer Vision task for the 6CS012 Artificial Intelligence and Machine Learning assessment. The project focused on classifying 17,229 flower images from 19 different classes using deep learning techniques. Three models were developed and compared: a Baseline CNN, a Deeper CNN with Batch Normalization and Dropout regularization, and a MobileNetV2 transfer learning model.

The models were evaluated using validation accuracy, loss, precision, recall, F1-score, confusion matrices, and sample predictions. Additional experiments were also performed to compare Adam and SGD optimizers and to analyze the effect of removing Dropout. The Baseline CNN achieved a validation accuracy of 77.7%, while the Deeper CNN improved performance to 81.0%. The MobileNetV2 transfer learning model achieved the best performance with a validation accuracy of 89.4%.

The results show that transfer learning and regularization techniques significantly improved classification performance and reduced overfitting. Overall, the project demonstrates the effectiveness of deep learning methods for flower image classification tasks.

Table of Contents

Abstract 2
1. Introduction 5
2. Dataset 5
  2.1 Dataset Description 5
  2.2 Dataset Visualizations 6
  2.3 Data Preprocessing 7
3. Methodology 8
  3.1 Baseline CNN Architecture 8
    3.1.1 Baseline Model Summary 8
  3.2 Deeper CNN Architecture with Regularization 8
    3.2.1 Deeper Model Summary 9
  3.3 Transfer Learning with MobileNetV2 9
    3.3.1 MobileNetV2 Transfer Learning Summary 9
4. Experiments and Results 11
  4.1 Hardware and Training Environment 11
  4.2 Overall Model Performance Comparison 11
  4.3 Detailed Evaluation Metrics 12
  4.4 Training Curves 12
  4.5 Confusion Matrix 14
  4.6 Sample Prediction Outputs 17
  4.7 Optimizer Analysis: Adam vs SGD 18
  4.8 Ablation Study: Effect of Removing Dropout 19
5. Challenges and Observations 21
  5.1 Overfitting 21
  5.2 Class Imbalance 21
  5.3 Training Time vs Accuracy Trade-off 21
  5.4 Hardware Acceleration 21
6. Conclusion and Future Work 22
7. References 23

Table of Figure

Figure 1. Distribution of images across all 19 flower classes showing class imbalance 6

Figure 2. Sample images from five representative flower classes in the dataset 6

Figure 3. Visualization of data augmentation techniques applied to training images (rotation, flip, zoom, shift) 7

Figure 4. Baseline CNN — Training and Validation Accuracy (left) and Loss (right) over epochs 12

Figure 5. Deeper CNN — Training and Validation Accuracy (left) and Loss (right) over epochs 13

Figure 6. MobileNetV2 Transfer Learning — Training and Validation Accuracy (left) and Loss (right) over epochs (Phase 1 and Phase 2 combined) 13

Figure 7. Confusion matrix showing per-class prediction performance for MobileNetV2 across all 19 flower classes 14

Figure 8. Confusion matrix for Baseline CNN across all 19 flower classes..... 15

Figure 9. Confusion matrix for Deeper CNN across all 19 flower classes..... 16

Figure 10. Sample test images with predicted labels and confidence scores from MobileNetV2 (Transfer Learning) model 17

Figure 11. Sample test images with predicted labels and confidence scores from Baseline CNN17

Figure 12. Sample test images with predicted labels from the Deeper CNN model. 18

Figure 13. Training and validation accuracy comparison between Adam and SGD optimizers on the Deeper CNN..... 19

Figure 14. Training vs validation accuracy curves for deeper CNN with Dropout (left) and without Dropout (right)..... 20

1. Introduction

Image classification is one of the most widely used applications of deep learning and computer vision. It involves training a model to recognize and classify images into different categories. This technology is useful in areas such as agriculture, plant disease detection, biodiversity monitoring, and botanical research.

Convolutional Neural Networks (CNNs) are commonly used for image classification because they can automatically learn important image features such as shapes, textures, edges, and colours without manual feature extraction (freeCodeCamp.org, 2020).

In this project, three different models were developed and compared for flower image classification: a baseline CNN, a deeper CNN with regularization, and a MobileNetV2 transfer learning model. All models were trained and evaluated on 19 flower classes to compare their performance and understand the effect of deeper architectures and transfer learning.

2. Dataset

2.1 Dataset Description

The dataset used for this project was a collection of flower images organized into 19 classes, downloaded from Kaggle. The dataset contained a total of 17,229 images stored in separate folders by class. The 19 flower classes included commonly known flowers such as tulip, sunflower, iris, rose, daisy, orchid, lavender, lotus, and others.

The dataset was slightly imbalanced across classes. Some classes such as iris, tulip, and sunflower had over 1,000 images, while other classes like orchid, lavender, and lotus had fewer samples. This imbalance affected per-class performance, as the model had more examples for some flower types than others.

2.2 Dataset Visualizations

The dataset contains 19 flower classes with a total of 17,229 images. Most classes contain around 900–1,050 images, while classes such as orchid, lavender, and lotus have fewer samples. This shows a slight class imbalance in the dataset, which may affect model performance on minority classes.

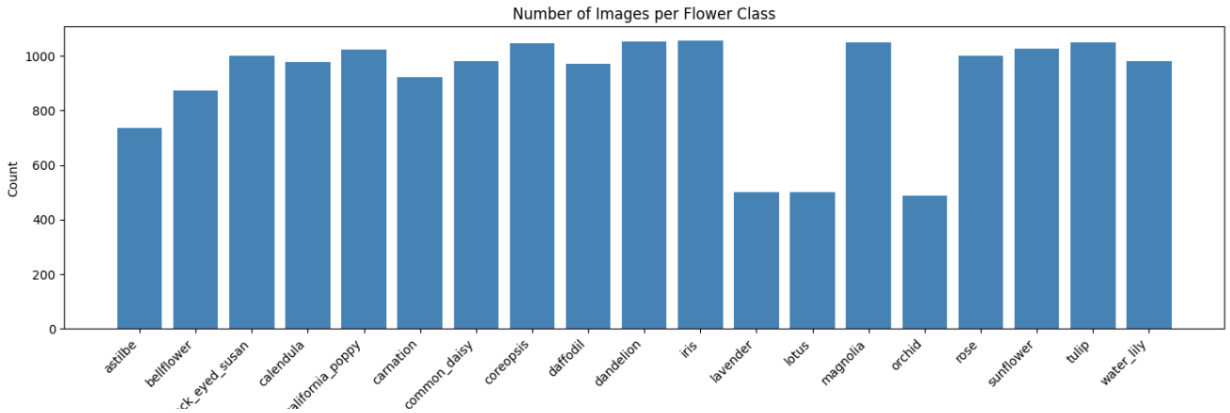

Figure 1. Distribution of images across all 19 flower classes showing class imbalance

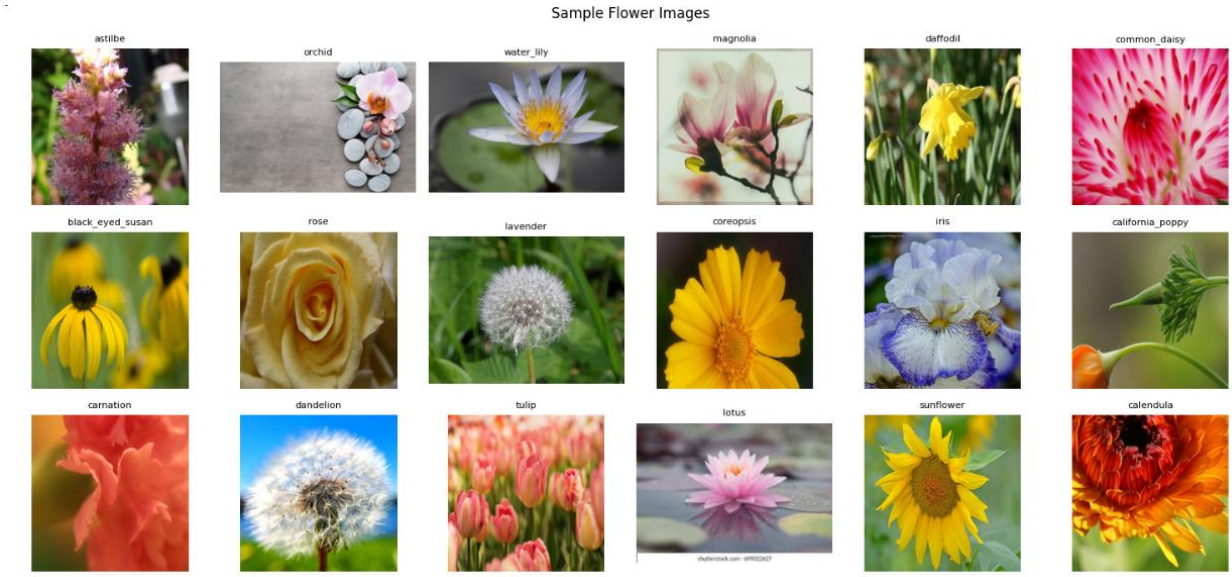

Figure 2. Sample images from five representative flower classes in the dataset

Figure 3. Visualization of data augmentation techniques applied to training images (rotation, flip, zoom, shift)

The figure shows examples of augmented training images generated using rotation, zooming, shifting, and flipping techniques. Data augmentation helps increase dataset diversity and reduces overfitting during training.

2.3 Data Preprocessing

Before training, corrupted or unreadable images were removed from the dataset. All images were resized to 128 × 128 pixels for the CNN models and 224 × 224 pixels for the MobileNetV2 model. Pixel values were normalized from 0–255 to 0–1 to improve training performance.

Data augmentation was applied only to the training images using Keras ImageDataGenerator. Techniques such as rotation, flipping, zooming, and shifting were used to create different variations of images. This helped reduce overfitting and improved model generalization. Validation images were kept unchanged for proper evaluation.

3. Methodology

3.1 Baseline CNN Architecture

The baseline CNN was designed with three convolutional blocks, each consisting of a convolutional layer followed by a max pooling layer. The first block used 32 filters with a 3×3 kernel, the second used 64 filters, and the third used 128 filters. All convolutional layers used the ReLU activation function. After the convolutional blocks, the feature maps were flattened and passed through three fully connected dense layers with 256, 128, and 64 neurons respectively. The final output layer used 19 neurons with softmax activation. The model was compiled with the Adam optimizer and categorical cross-entropy loss.

3.1.1 Baseline Model Summary

Layer (Type)	**Output Shape**	**Parameters**
Conv2D (32 filters, 3×3, ReLU)	(None, 126, 126, 32)	896
MaxPooling2D (2×2)	(None, 63, 63, 32)	0
Conv2D (64 filters, 3×3, ReLU)	(None, 61, 61, 64)	18,496
MaxPooling2D (2×2)	(None, 30, 30, 64)	0
Conv2D (128 filters, 3×3, ReLU)	(None, 28, 28, 128)	73,856
MaxPooling2D (2×2)	(None, 14, 14, 128)	0
Flatten	(None, 25088)	0
Dense (256, ReLU)	(None, 256)	6,422,784
Dense (128, ReLU)	(None, 128)	32,896
Dense (64, ReLU)	(None, 64)	8,256
Dense (19, Softmax) – Output	(None, 19)	1,235
Total Trainable Parameters		~6,558,419

Table 3: Baseline CNN Model Summary

3.2 Deeper CNN Architecture with Regularization

The deeper CNN model was developed by extending the baseline CNN with more layers. The model used 6 convolutional blocks, which is double the number used in the baseline model. Each block contained two convolutional layers followed by a max pooling layer.

Batch Normalization was added after convolutional layers to make training more stable and faster. Dropout layers were also added after pooling and fully connected layers to reduce overfitting. A

dropout rate of 0.25 was used in convolutional blocks and 0.5 in fully connected layers. These regularization techniques helped improve the model's generalization performance (freeCodeCamp.org, 2020; Krish Naik, 2021).

3.2.1 Deeper Model Summary

Layer (Type)	**Output Shape**	**Parameters**
Conv2D (32, 3×3, ReLU) × 2 + BatchNorm	(None, 128, 128, 32)	10,272
MaxPooling2D + Dropout (0.25)	(None, 64, 64, 32)	0
Conv2D (64, 3×3, ReLU) × 2 + BatchNorm	(None, 64, 64, 64)	55,680
MaxPooling2D + Dropout (0.25)	(None, 32, 32, 64)	0
Conv2D (128, 3×3, ReLU) × 2 + BatchNorm	(None, 32, 32, 128)	221,952
MaxPooling2D + Dropout (0.25)	(None, 16, 16, 128)	0
Flatten + Dense (512, ReLU) + Dropout (0.5)	(None, 512)	16,779,776
Dense (256, ReLU) + Dropout (0.5)	(None, 256)	131,328
Dense (19, Softmax) – Output	(None, 19)	4,883
Total Trainable Parameters		17,202,419

Table 4: Deeper CNN Model Summary

3.3 Transfer Learning with MobileNetV2

MobileNetV2 was selected as the pretrained model for transfer learning. It was originally trained on the ImageNet dataset containing over 1 million images from 1,000 categories, allowing it to learn useful image features for classification tasks (Sandler et al., 2018).

The transfer learning process was completed in two phases. In Phase 1, all MobileNetV2 base layers were frozen and only the new classification head was trained. The classification head included a GlobalAveragePooling2D layer, a Dense layer with 256 neurons and ReLU activation, a Dropout layer with a rate of 0.4, and a final Dense layer with 19 neurons and softmax activation. In Phase 2, the last 30 layers of MobileNetV2 were unfrozen and fine-tuned using a very low learning rate of 0.00001 to improve performance and reduce overfitting (Sentdex, 2019).

3.3.1 MobileNetV2 Transfer Learning Summary

Component	Details
Base Model	MobileNetV2 (pretrained on ImageNet)
Input Size	224 × 224 × 3
Phase 1 – Frozen Layers	All MobileNetV2 base layers frozen
Phase 2 – Unfrozen Layers	Last 30 layers fine-tuned
Classification Head	GlobalAveragePooling2D → Dense(256, ReLU) → Dropout(0.4) → Dense(19, Softmax)
Phase 1 Learning Rate	0.001 (Adam)
Phase 2 Learning Rate	0.00001 (Adam)
Total Base Parameters	2,257,984
Total Parameters	2,590,803
Trainable Parameters	332,819
Non-Trainable Parameters	2,257,984

Table 5: MobileNetV2 Transfer Learning Configuration

4. Experiments and Results

4.1 Hardware and Training Environment

Component	Details
Platform	Google Colaboratory
Hardware Accelerator	NVIDIA T4 GPU
TensorFlow Version	2.x (latest Colab default)
RAM Available	8 GB
Baseline CNN – Training Time	41.2 minutes
Deeper CNN – Training Time	44.8 minutes
MobileNetV2 – Total Training Time	94.2 minutes

Table 6: Hardware and Training Time Details

All models were trained using Google Colab with NVIDIA T4 GPU acceleration. The baseline CNN required the least training time, while the MobileNetV2 transfer learning model required the longest training time due to two-phase training and fine-tuning. GPU acceleration significantly reduced overall training time and improved computational efficiency.

4.2 Overall Model Performance Comparison

Model	Val. Accuracy	Val. Loss	Training Time (approx.)	Key Feature
Baseline CNN	77.7%	0.7747	41.2 min	3 conv blocks, no regularization
Deeper CNN	81.0%	0.6464	44.8 min	6 conv blocks, BatchNorm, Dropout
MobileNetV2 (Transfer Learning)	89.4%	0.3653	94.2 min	Transfer Learning (ImageNet pre-training + fine-tuning)

Table 1: Summary of Model Accuracy Comparison

The MobileNetV2 transfer learning model achieved the highest validation accuracy and lowest validation loss among all models. The deeper CNN performed better than the baseline CNN due to the use of Batch Normalization and Dropout. However, the transfer learning model required the longest training time because of two-phase fine-tuning.

4.3 Detailed Evaluation Metrics

Each model was evaluated using accuracy, precision, recall, and F1-score across all 19 classes. The following table presents the weighted average metrics for all three models.

Model	Accuracy	Precision (Weighted)	Recall (Weighted)	F1-Score (Weighted)
Baseline CNN	77.7%	78%	77%	77%
Deeper CNN	81.0%	81%	81.0%	80%
MobileNetV2 (Transfer Learning)	89.4%	90%	89%	89%

Table 7: Classification Metrics — Weighted Average Across 19 Classes

The MobileNetV2 transfer learning model achieved the best overall performance across all evaluation metrics. The deeper CNN also improved accuracy and F1-score compared to the baseline CNN due to additional convolutional layers and regularization techniques. These results show that transfer learning and regularization significantly improved classification performance and model generalization.

4.4 Training Curves

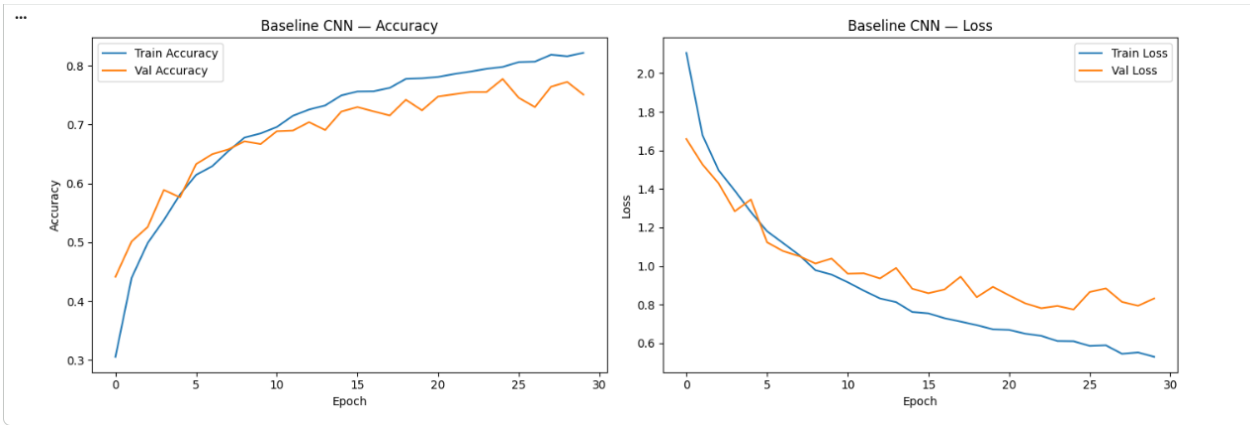

Figure 4. Baseline CNN — Training and Validation Accuracy (left) and Loss (right) over epochs

The baseline CNN model showed gradual improvement in training and validation accuracy over the training epochs, while the loss steadily decreased. The validation curves remained relatively close to the training curves, indicating that the model learned useful image features with limited overfitting. However, the overall performance was lower compared to the deeper CNN and transfer learning models.

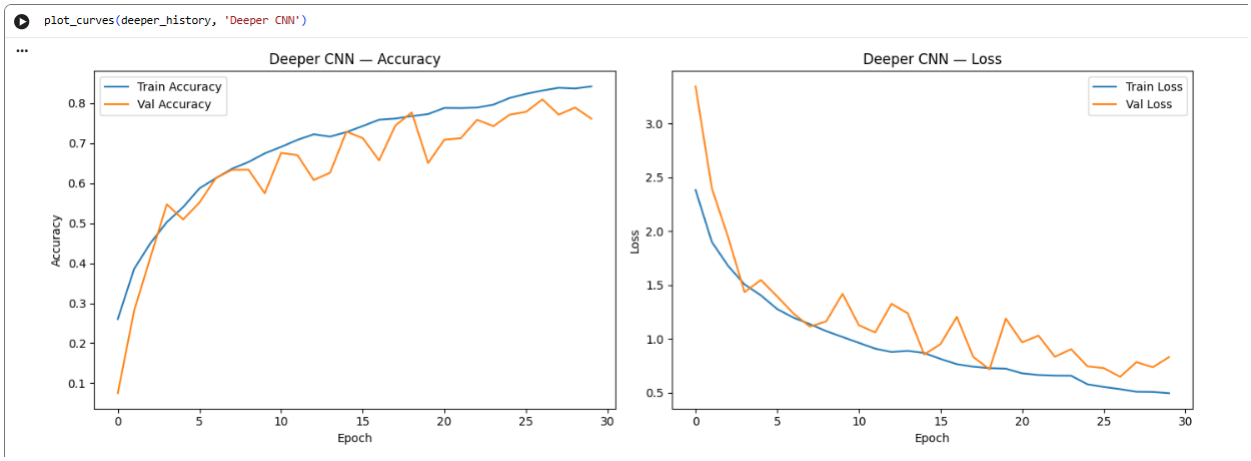

Figure 5. Deeper CNN — Training and Validation Accuracy (left) and Loss (right) over epochs

The deeper CNN model showed steady improvement in training and validation accuracy during training, while the loss decreased over epochs. Compared to the baseline CNN, the deeper model achieved better validation performance due to the use of additional convolutional layers, Batch Normalization, and Dropout. The curves also indicate reduced overfitting and improved learning stability.

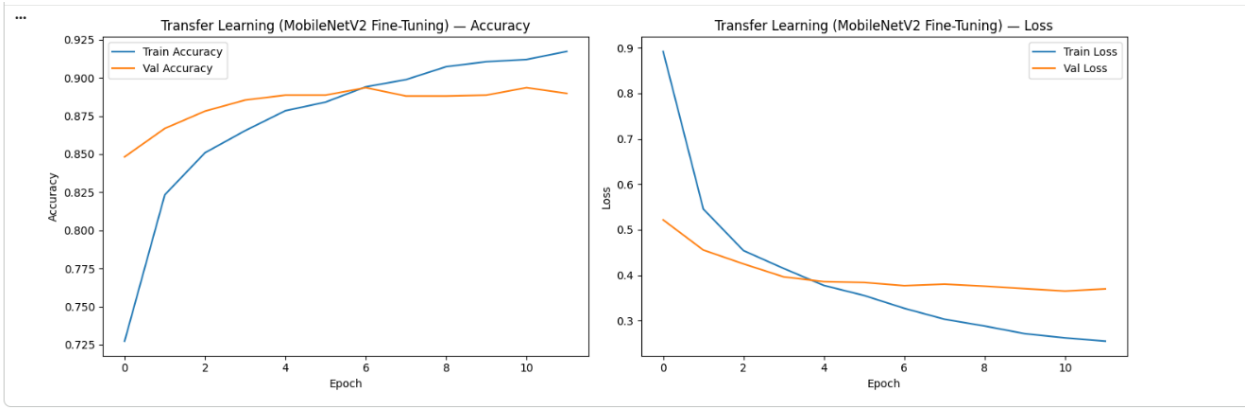

Figure 6. MobileNetV2 Transfer Learning — Training and Validation Accuracy (left) and Loss (right) over epochs (Phase 1 and Phase 2 combined)

The MobileNetV2 transfer learning model achieved the highest validation accuracy and the lowest validation loss among all models. The training curves remained stable throughout both training phases, showing effective feature learning and strong generalization performance with limited overfitting.

4.5 Confusion Matrix

A confusion matrix was used to evaluate how well the models classified each flower category. It helps identify correct predictions and shows which flower classes were commonly misclassified.

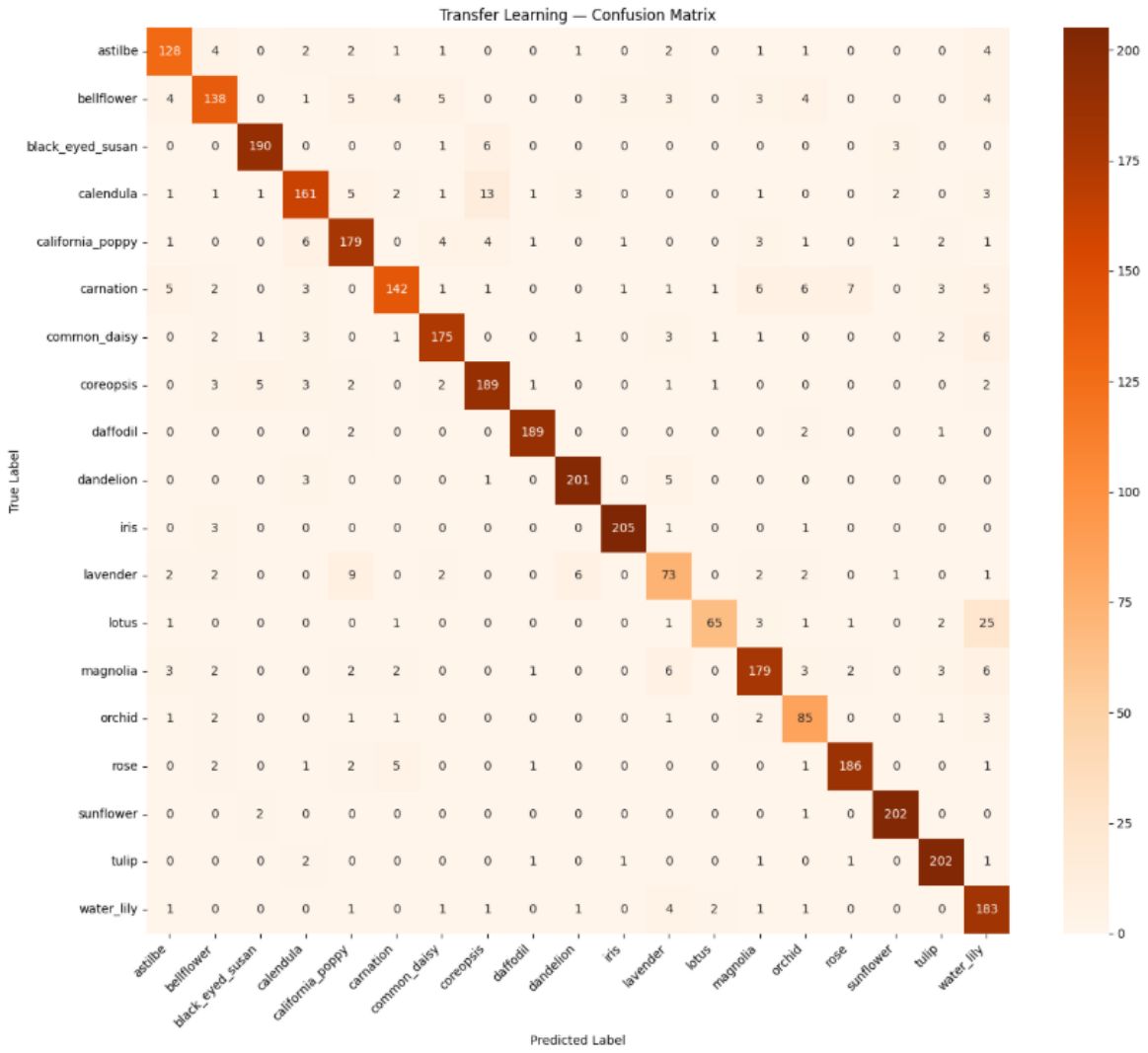

Figure 7. Confusion matrix showing per-class prediction performance for MobileNetV2 across all 19 flower classes

Figure 7 shows the confusion matrix for the MobileNetV2 model across all 19 flower classes. Most predictions are concentrated along the diagonal, indicating that the model correctly classified the majority of images. Some confusion occurred between visually similar flower classes such as lotus and water_lily, but overall the model achieved strong classification performance with high accuracy across most categories.

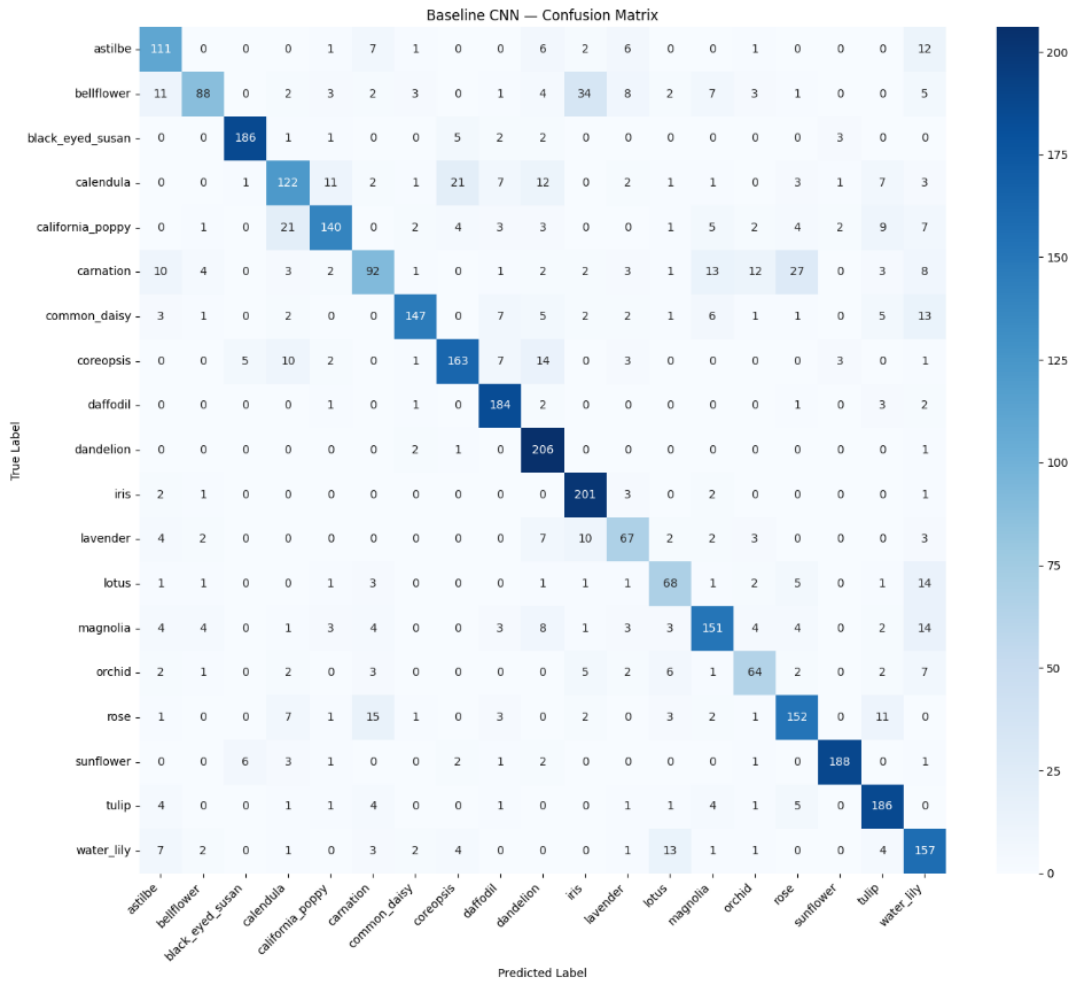

Figure 8. Confusion matrix for Baseline CNN across all 19 flower classes

The Baseline CNN confusion matrix shows that the model correctly classified many flower images, but more misclassifications are visible compared to MobileNetV2. Errors mainly occurred between visually similar flower categories, indicating that the baseline model learned fewer discriminative features than the transfer learning model.

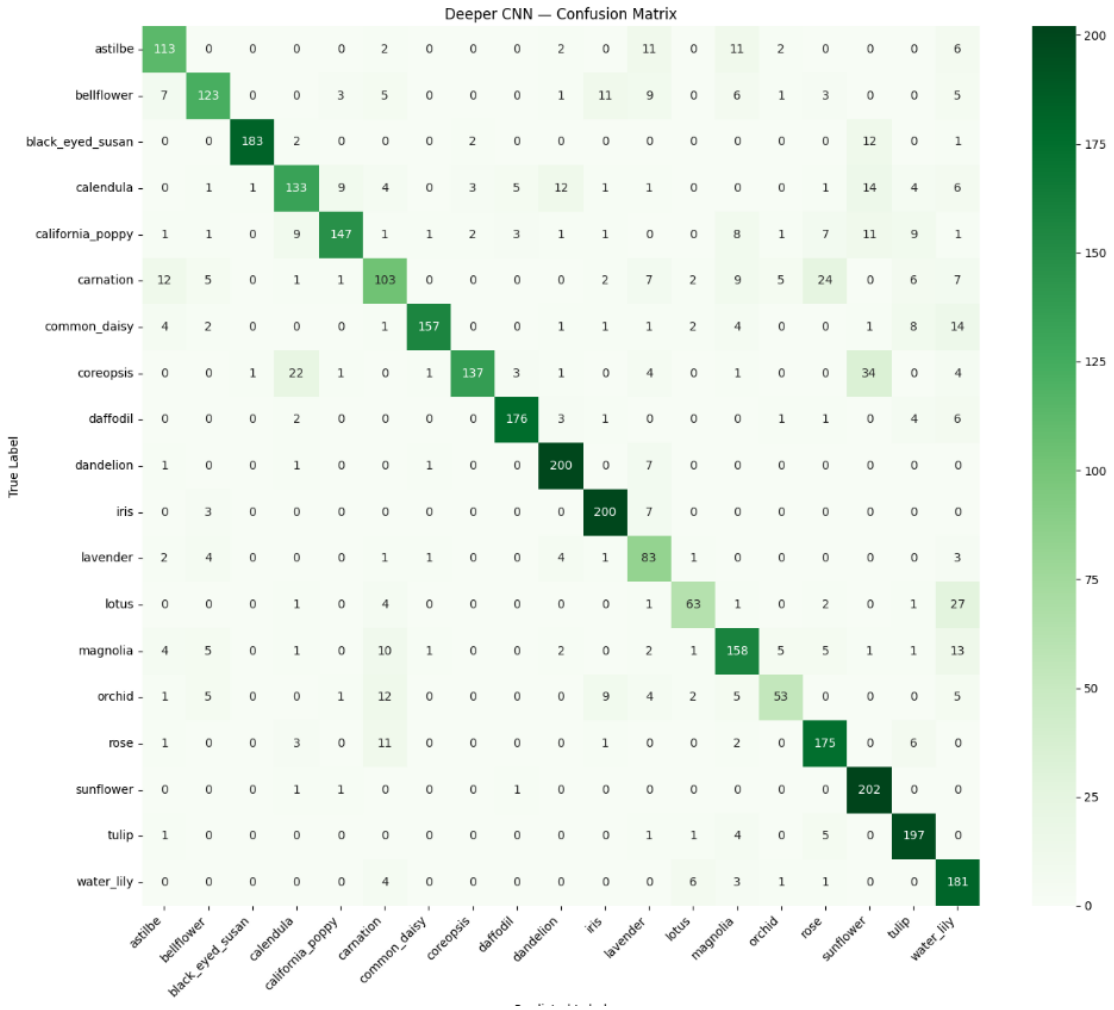

Figure 9. Confusion matrix for Deeper CNN across all 19 flower classes

Figure 9 shows the confusion matrix for the Deeper CNN model. Compared to the Baseline CNN, fewer misclassifications were observed, indicating improved feature extraction due to additional convolutional layers and regularization techniques such as Batch Normalization and Dropout.

4.6 Sample Prediction Outputs

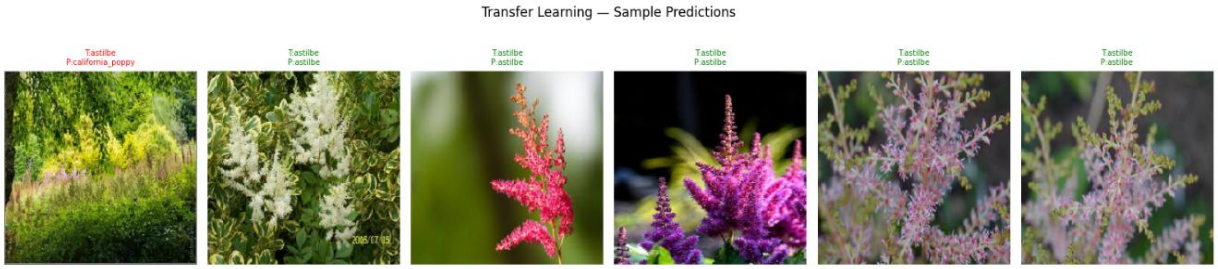

Figure 10. Sample test images with predicted labels and confidence scores from MobileNetV2 (Transfer Learning) model

The sample predictions demonstrate that the MobileNetV2 model correctly classified most flower images with high confidence. Correct predictions are shown in green, while incorrect predictions are shown in red. This further confirms the strong generalization performance of the transfer learning approach.

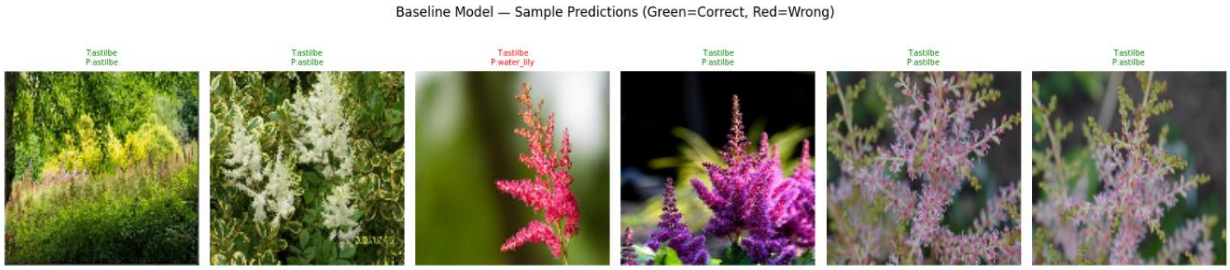

Figure 11. Sample test images with predicted labels and confidence scores from Baseline CNN

The sample predictions from the Baseline CNN model show that several flower images were classified correctly, while some visually similar classes were misclassified. Incorrect predictions are highlighted in red, indicating the limitations of the baseline architecture compared to the deeper CNN and transfer learning models.

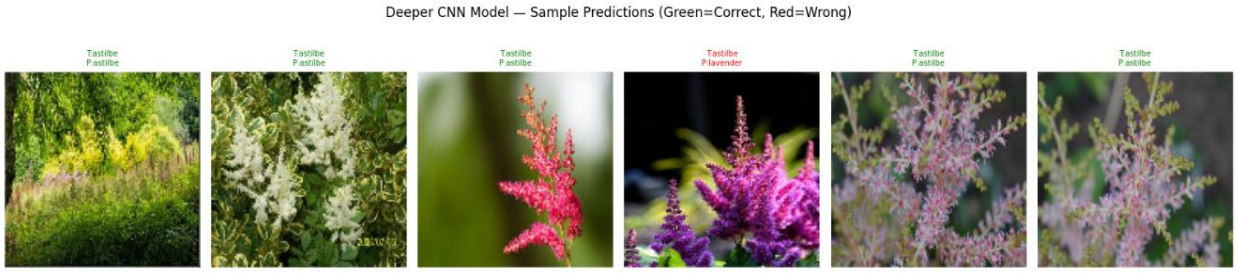

Figure 12. Sample test images with predicted labels from the Deeper CNN model.

The sample predictions from the Deeper CNN model show improved classification performance compared to the Baseline CNN model. Most flower images were correctly identified, although a few visually similar classes were still misclassified. This demonstrates that the deeper architecture with Batch Normalization and Dropout improved feature learning and generalization performance.

4.7 Optimizer Analysis: Adam vs SGD

An experiment was conducted to compare the Adam and SGD optimizers using the Deeper CNN model. The Adam optimizer achieved a higher validation accuracy of 81.0% and converged faster during training. In comparison, SGD with a learning rate of 0.01 and momentum of 0.9 achieved a lower validation accuracy of 75.9% and required more epochs to stabilize. The graph also shows that Adam learned more efficiently and reached better performance earlier than SGD. Therefore, Adam was selected as the final optimizer for this flower classification task because it provided better accuracy and faster convergence (TensorFlow, 2023).

Optimizer	**Validation Accuracy**	**Convergence Speed**	**Notes**
Adam	81.0%	Fast (~15 epochs to plateau)	Adaptive LR, less manual tuning
SGD (lr=0.01, momentum=0.9)	75.9%	Slow (~25+ epochs)	Requires careful LR tuning

Table 8: Optimizer Comparison on Deeper CNN

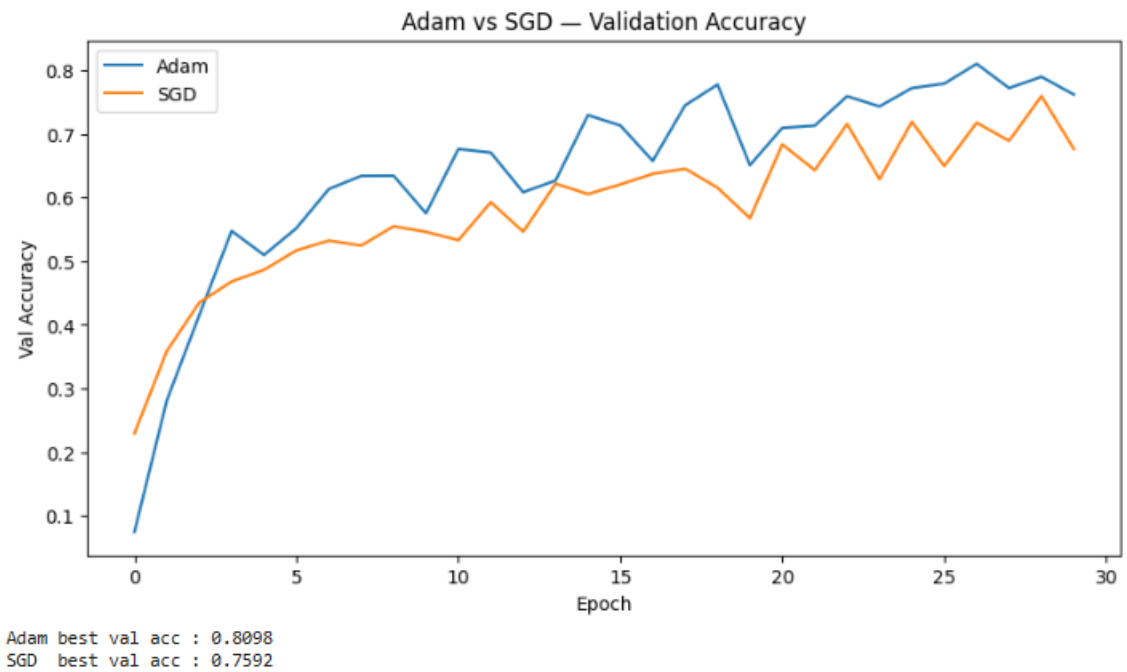

Figure 13. Training and validation accuracy comparison between Adam and SGD optimizers on the Deeper CNN

4.8 Ablation Study: Effect of Removing Dropout

An ablation study was conducted to evaluate the effect of Dropout regularization in the Deeper CNN model. The model with Dropout achieved better generalization performance and more stable validation accuracy during training. In contrast, the model without Dropout showed signs of overfitting, where training accuracy increased rapidly while validation performance became less stable. These results demonstrate that Dropout helped reduce overfitting and improved the model's ability to generalize to unseen flower images.

Configuration	Train Accuracy	Val. Accuracy	Observation
Deeper CNN with Dropout	84.2%	81.0%	Better generalization, reduced overfitting
Deeper CNN without Dropout	91.6%	81.3%	Higher training accuracy but unstable validation performance

Table 9: Ablation Study — Effect of Dropout on Model Performance

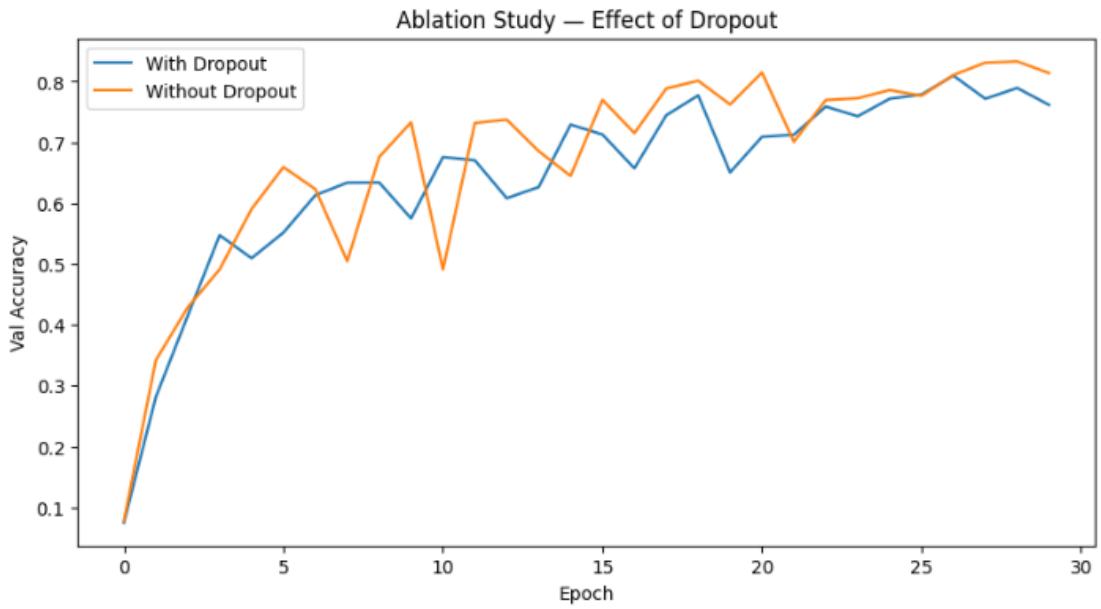

Figure 14. Training vs validation accuracy curves for deeper CNN with Dropout (left) and without Dropout (right)

5. Challenges and Observations

5.1 Overfitting

Overfitting was mainly seen in the Baseline CNN model. The training accuracy became much higher than the validation accuracy, which means the model was learning the training images too well but was not performing as well on new images. To solve this problem, Batch Normalization and Dropout were added in the Deeper CNN model. These techniques helped the model generalize better and improved validation performance.

5.2 Class Imbalance

The dataset had a small imbalance because some flower classes had more images than others. Classes such as iris and tulip performed better because they had more training samples, while classes such as orchid and lavender were harder to classify correctly. Some flowers also looked visually similar, which caused confusion between classes. In future work, techniques such as class balancing or adding more images could help improve performance.

5.3 Training Time vs Accuracy Trade-off

A trade-off was observed between training time and accuracy. The Baseline CNN trained faster and took about 41.2 minutes, but it achieved the lowest accuracy of 77.7%. The Deeper CNN took slightly longer to train at 44.8 minutes and improved the accuracy to 81.0%. The MobileNetV2 transfer learning model took the longest time, about 94.2 minutes, because it was trained in two phases. However, it achieved the best accuracy of 89.4%. This shows that more advanced models usually require more training time but can produce better results.

5.4 Hardware Acceleration

All experiments were performed using Google Colab with an NVIDIA T4 GPU. The GPU helped reduce training time and made it possible to train the models much faster. TensorFlow 2.x and Keras were used to build and train all models in this project. Without GPU support, training the deeper CNN and transfer learning models would have taken much longer.

6. Conclusion and Future Work

This project explored flower image classification using three different deep learning approaches: a Baseline CNN, a Deeper CNN with regularization, and a MobileNetV2 transfer learning model. The results showed that increasing model complexity improved classification performance. The Baseline CNN achieved reasonable accuracy, while the Deeper CNN improved performance by learning more detailed image features. The best results were obtained using MobileNetV2 transfer learning, which achieved the highest validation accuracy and better generalization across most flower classes.

The experiments also showed the importance of techniques such as Dropout, Batch Normalization, and optimizer selection. Adam optimizer converged faster and achieved better validation accuracy than SGD. The ablation study further demonstrated that removing Dropout increased overfitting and reduced model generalization.

Although the models performed well overall, some flower classes with similar visual patterns were still misclassified. In future work, performance could be improved by using a larger dataset, applying stronger data augmentation techniques, balancing class distributions, and experimenting with more advanced architectures such as EfficientNet or ResNet. Further hyperparameter tuning and longer fine-tuning could also improve accuracy.

Overall, the project demonstrated the effectiveness of deep learning and transfer learning techniques for multi-class flower image classification.

7. References

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). *MobileNetV2: Inverted residuals and linear bottlenecks*. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 4510–4520). https://doi.org/10.1109/CVPR.2018.00474

freeCodeCamp.org. (2020). *Convolutional Neural Networks – Full tutorial for beginners* [Video]. YouTube. https://www.youtube.com/watch?v=YRhxdVk_sIs

Sentdex. (2019). *Transfer learning and fine-tuning with MobileNet – Deep learning with Python, TensorFlow, and Keras* [Video]. YouTube. https://www.youtube.com/watch?v=19LLLaGLap0

Krish Naik. (2021). *Image classification using CNN with Keras – Complete end-to-end project* [Video]. YouTube. https://www.youtube.com/watch?v=7HPwo4wnJeA

TensorFlow. (2023). *Transfer learning and fine-tuning – TensorFlow Core tutorial* [Video]. YouTube. https://www.youtube.com/watch?v=oDHpqu52soI